YAZILIM ANALİZİ VE TASARIMI

BMH356

Gökalp Çınarer

HAFTA 1 : Giriş ve Temel Kavramlar

a) Tanımlar ve Kısaltmalar
b) Yazılım Analizi ve Tasarımına Giriş

Gökalp Çınarer



a) Temel Kavramlar

Değer

Üretilen ürün, teknoloji, hizmet, fikir, kültürdür. Günümüzde bilgi üretilmesi, bilinç üretilmesi, kültür üretilmesi, sanat ve felsefe üretilmesi, bilgelik üretilmesi katma değeri çok yüksek değerlerdir. Çünkü bunlar tarihi etkileyecek değerlerdir.

Paydaş

Süreçleri etkileyen ve süreçlerden etkilenen menfaat sahipleridir. Çalışanlar, yatırımcılar, kurucu sermayedarlar, hissedarlar, müşteriler, tedarikçiler, dış işlik alımları, iş ortakları, kamu kurum ve kuruluşları, bankalar ve finans kuruluşları, sendikalar, danışmanlar gibi tekneden ekmek yiyecek olanlardır.

Paydaş, yazılımın başarısını etkileyen veya başarısından etkilenen kişi ve kurumlardır.

Temel Kavramlar

İşlev (Fonksiyon)

Girdi olarak kabul edilen değişkenlerden bir çıktı değer üretilmesini sağlayan kurallardır.

Süreç Yönetimi

İstatistiksel teknikler kullanılarak ve sağlıklı iletişim ortamı kurularak, farklı açılardan bakan kişilerin aynı dili konuşmasının sağlanmasıdır.

İstatistik

İş süreçlerinin gözlenmesi, bilgilerin sistematik olarak toplanması ve işlenmesi sonucunda belirli duyarlılıkta risklere yönelik tahminde bulunmayı ve yorum yapmayı sağlayan bilim dalıdır.

Süreç: İşlevlerin planlanıp değer ya da ürün haline getirildiği önceden belirlenmiş adımlardan oluşan iş akışıdır.

Temel Kavramlar

Kalite

Süreçlerin işlevlerinde toplanan verilere dayalı gerçekçi karar verme ve sürekli iyileştirme aşamalarında hataların temelinde yatan değişkenliğin belirlenmesi, azaltılması veya belli sınırlar dahilinde tutulmasıdır. Kalitenin temel kuramları olan ölçüm ve analize dayalı, sorgulama ve kıyaslama yapılarak hataların önlenmesi, maliyetlerin düşürülmesi, verimin artırılması hedeflenir.

Değişkenlik

Tüm olayların temelinde değişkenlikler vardır. Hataların, belirsizliklerin ve eksik verilerin çok büyük bir bölümü de değişkenlikten kaynaklanır. İstatistiksel teknikler kullanılarak değişkenliğin özellikleri incelenmeli ve hataların kaynakları tespit edilmelidir.

Toplam Kalite Yönetimi

İş üretilirken insan, sistem, yönetim ve ürün kalitesinin bir arada sürekli olarak geliştirilmesi, planlanması, kontrol çalışmalarının yapılması ve standartların oluşturulmasıdır.

Sürekli İyileştirme - Kaizen

Sürekli eğitim, sürekli bilgi ve beceri kazanma, sürekli araştırma ve geliştirme çalışmalarının yapılmasıdır. Organizasyonda iyileştirme çalışmalarının sürekli olması gerekir.

Temel Kavramlar

İstatistiksel Süreç Kontrolü

Organizasyonda sürekli gelişme sağlamak ve kaliteyi artırmak için iş üretilirken kontrol teknikleri için istatistiksel verilerden yararlanmaktır.

Dışarıdan Hizmet Alım (Dış İşlik)

Organizasyonun her işi kendisi yapmak yerine asıl faaliyet alanı dışındaki işleri dış firmalara yaptırmasıdır.

Benchmarking - Kıyaslama

Süreçlerde geliştirilen stratejilerin, örgüt yapısının ve tüm uygulamaların kıyaslanarak en iyisini bulmaya çalışmaktır. Mükemmel ve en iyisi olmak için en mükemmelini ve en iyisini aramaktır. Ararken değişilir, en iyi ve en mükemmele dönüşülür.

Esnek Üretim

Ürünün istenildiği zamanda üretilmesi için gerekli planların hazırlanmasıdır. Stok üretim yerine, istenilen miktarda istenilen zamanda üretmektir.

Tam zamanında üretim – Just In Time.

Temel Kavramlar

Yalın Organizasyon Model

Organizasyon yapısının sadeleştirilmesi ve basitleştirilmesidir. Dikey organizasyon yerine yatay organizasyon yapısının oluşturulmasıdır.

Otomasyon

Rutin işlerin mümkün olduğu ölçüde robotlara ve bilgisayar kontrollü makinelere yaptırılmasıdır.

Kademe Azaltma

Organizasyonda işe karar veren ile uygulayan arasındaki kademelerin mümkün olduğu ölçüde ortadan kaldırılmasıdır. Gereksiz ve katma değer yaratmayan kademelerin, bileşenlerin ve iş süreçlerinin kaldırılmasıdır.

Sürekli Eğitim

İşe uygun eğitimli ve bilgili eleman alınmakla yetinilmemesi; sürekli eğitime, bilgi ve beceri kazandırmaya önem verilmesidir.

Temel Kavramlar

Otokontrol

Ekip içinde çalışanların birbirlerini kontrol etmeleridir.

Toplam Verimli Bakım

Malzeme, enerji, ekipman ve personel ile ilgili kayıpların ortadan kaldırılması için bakım ve onarım çalışmalarının planlı ve düzenli bir şekilde yapılmasıdır.

Çalışanları Güçlendirme

Çalışanlara yetki ve sorumluluk devredilmesi ve bu sayede çalışanların karar alma sürecine katılmasıdır. Ekip çalışmasına önem verilmesidir. Çalışanların motivasyonu ve ödüllendirilmesidir. Çalışanların organizasyonda pay sahibi olmasıdır.

Kurumsal Anlayış

Makama, yeteneğe resmiyet içeren etkili bir yönetim biçimidir.

Temel Kavramlar

Kurumsal Hafıza

Geçmişi hatırlamak amacıyla, üretilen değerler ile birlikte iş süreçleri ve tecrübelerin saklanması ve süreçten sürece bilinçlenerek aktarılmasıdır. Belgeler aracılığıyla bilgi ve fikir birikimi elde edinebilmektir. Kurumun geçmişi ile geleceği arasında ortak bir hafızanın gelişmesine katkıda bulunmak amaçlanır.

Kısaltmalar

SDL: Security Development Lifecycle

TSP: Team Software Process

CMM: Capability Maturity Model (maturity: olgunluk, ergenlik)

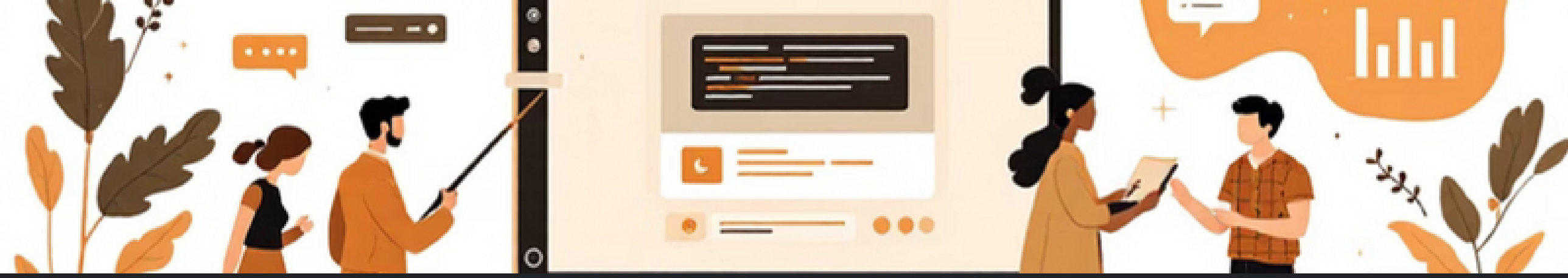
1.b) Yazılım Analizi ve Tasarımına Giriş

Yazılım, en basit tanımıyla, bir bilgisayarın veya elektronik cihazın ne yapacağını belirleyen talimatlar, veriler ve programlar bütünüdür;

Çeşitli ölçeklerdeki donanımlarda işleyen talimat dizileri, algoritmalar ve veri yapılarını kapsar, aynı zamanda geliştirme sürecinde veya son kullanıcıya sunulan belgeler, kılavuzlar ve arayüz tasarımları gibi yardımcı unsurları da içerir.



Gökalp Çınarer



Yazılımı benzersiz kılan, onun bir "ürün" olmanın ötesinde, diğer sistemleri üretmek için bir "araç" olarak da işlev görmesidir. Örneğin, bir grafik tasarım programı yeni görseller yaratırken, bir veritabanı yönetim aracı milyonlarca veri setini organize eder. Fiziksel bir varlığı olmayan yazılım, mekanik aşınmaya maruz kalmaz; ancak zamanın akışıyla teknolojik ilerlemeler, kullanıcı alışkanlıkları veya dış etkenler karşısında "eskiyen" bir nitelik kazanır, yani yetersizleşir ve revizyona ihtiyaç duyar.

Yazılımın yaşam döngüsü tam da bu yolculuğu ifade eder konsept aşamasından başlayarak geliştirme, dağıtım, kullanım ve nihayetinde emekliliğe kadar uzanan, sürekli bir döngü. Bu döngüde değişim kaçınılmazdır; her güncelleme veya adaptasyon, beklenmedik yan etkiler doğurabilir, örneğin yeni bir özellik eklerken eski modüllerin uyumsuzlaşması gibi, ve bu da yazılımın güvenilirliğini sürekli bir denge oyunu haline getirir.

Gökalp Çınarer

b.1) Bir Ürünün Değer Üretme Yolculuğu:

İhtiyaç, Fikir

1

Her şey bir sorunu, bir eksikliği veya karşılanmamış bir talebi fark etmekle başlar. Bu adım, "İnsanların neye ihtiyacı var?" veya "Hangi süreci daha iyi hale getirebiliriz?" sorularına cevap arar. Fikir, bu ihtiyaca yönelik ilk çözüm önerisidir.

Örnek: İnsanlar ya bitkilerini sulamayı unutuyor (ihtiyaç) ya da gereğinden fazla su kullanarak israf yapıyor (ihtiyaç). Fikir: Toprağın nemini ölçen bir sensör ve hava durumu verilerini internetten çeken bir cihaz geliştirerek sadece gerektiğinde ve gerektiği kadar sulama yapan bir otomasyon sistemi kurmak.

Araştırma, Analiz (Teknoloji, Pazar)

2

Fikrin hayata geçirilebilir olup olmadığını anlamak için yapılan adımdır. Piyasada benzer ürünler var mı? Rakipler kimler? Hangi teknolojiler (sensörler, işlemciler, yazılım dilleri) kullanılabilir ve maliyetleri nedir? Hedef kitle kim ve bu ürüne para öderler mi?

Örnek: Piyasadaki mevcut sulama sistemleri araştırılır. Çoğunun sadece zamanlayıcıya sahip olduğu, toprağın gerçek nemini ölçmediği görülür. Wi-Fi modülleri, nem sensörleri ve mobil uygulama geliştirme platformlarının maliyetleri analiz edilir. Hedef pazarın evinde bitki yetiştirenler ve küçük hobi bahçesi sahipleri olduğuna karar verilir.

Tasarım, Simülasyon

3

Fikrin kağıt üzerinde veya dijital ortamda somutlaştırıldığı aşamadır. Ürünün teknik çizimleri, elektronik devre şemaları, mobil uygulamanın arayüz (UI/UX) taslakları hazırlanır. Simülasyonlar ile sistemin farklı koşullarda nasıl davranacağı test edilir.

Örnek: Cihazın suya dayanıklı dış kasası için 3D modeller (Tasarım) oluşturulur. Mobil uygulamanın "sulama geçmişi", "manuel sulama" gibi ekranları tasarlanır. Bilgisayar ortamında, "yağmur yağdığında sistemin sulamayı otomatik olarak iptal etmesi" gibi senaryolar simüle edilir.

b.1) Bir Ürünün Değer Üretme Yolculuğu:

Prototip Üretme ve Test

1

Tasarımın ilk fiziksel ve işlevsel örneği üretilir. Bu prototip, genellikle mükemmel görünmez ama temel fonksiyonları yerine getirmelidir. Amaç, teorideki tasarımın pratikte çalışıp çalışmadığını laboratuvar ortamında görmektir.

Örnek: 3D yazıcıdan basılmış bir kasa, lehimlenmiş devreler ve temel bir yazılımla ilk "Akıllı Sulama Sistemi" prototipi birleştirilir. Bir saksıya yerleştirilerek test edilir: Toprak kuruyunca suyu açıyor mu? Yeterince nemlenince kapatıyor mu?

Gerçek Saha Testi

2

Prototipin laboratuvardan çıkarılıp gerçek dünya koşullarında test edildiği kritik bir aşamadır. Beklenmedik sorunlar (güneş ışığı, yağmur, internet kesintisi vb.) bu aşamada ortaya çıkar.

Örnek: 5-10 adet prototip, farklı iklimlerde yaşayan arkadaş ve aile üyelerinin bahçelerine kurulur. Gelen geri bildirimler: "Cihazın plastik kasası direkt güneşte bir ay sonra gevrekleşti.", "Sabah çiyi, sensörün toprağı nemli sanmasına neden oluyor."

Ön Ürün Üretme

3

Saha testlerinden elde edilen geri bildirimlerle iyileştirilmiş, seri üretime çok yakın bir versiyonun küçük bir parti (örneğin 50-100 adet) halinde üretilmesidir. Amaç, üretim bandının ve montaj süreçlerinin sorunsuz işleyip işlemediğini test etmektir.

Örnek: UV ışınlarına dayanıklı yeni bir plastik malzeme seçilir ve sensör yazılımı çiy sorununu giderecek şekilde güncellenir. Bu yeni tasarımla 100 adetlik bir ön üretim yapılır.

b.1) Bir Ürünün Değer Üretme Yolculuğu:

Ürün Mükemmelleştirme

1

Ön üretimden ve daha geniş bir kullanıcı grubundan gelen geri bildirimlerle ürün üzerindeki son rötuşların yapılmasıdır. Bu, bir yazılım güncellemesi, ambalaj tasarımının değiştirilmesi veya kullanım kılavuzunun daha anlaşılır hale getirilmesi olabilir.

Örnek: Ön ürün kullanıcıları, mobil uygulamanın kurulumunun zor olduğunu belirtir. Bunun üzerine uygulama arayüzü basitleştirilir ve kutu içine daha anlaşılır bir "Hızlı Kurulum Kılavuzu" eklenir.

Test

2

Piyasaya sürülmeye hazır nihai ürünün kalite kontrol (QA) sürecidir. Ürünün tüm fonksiyonlarının hatasız çalıştığından, dayanıklılığından, güvenliğinden ve belirtilen tüm özellikleri karşıladığından emin olmak için kapsamlı ve sistematik testler yapılır.

Örnek: Nihai ürünler suya dayanıklılık testine sokulur, Wi-Fi bağlantısı farklı modemlerle test edilir, mobil uygulama onlarca farklı telefon modelinde denenir.

Müşteri Teslim

3

Ürünün pazara sunulması ve müşteriye ulaştırılmasıdır. Satış kanallarının (web sitesi, perakende mağazalar) belirlenmesi, lojistik, stok yönetimi ve ödeme sistemlerinin hazır olması bu adımı kapsar.

Örnek: Ürün, kendi web sitesi ve popüler e-ticaret platformları üzerinden satışa çıkarılır. Siparişler alınır, paketlenir ve kargo ile müşterilere gönderilir.

b.1) Bir Ürünün Değer Üretme Yolculuğu:

Eğitim

1

Müşterinin ürünü doğru ve verimli bir şekilde kullanabilmesi için gerekli bilgi ve materyallerin sağlanmasıdır. Kullanım kılavuzları, online videolar, Sıkça Sorulan Sorular (SSS) bölümü ve müşteri hizmetleri bu kapsamdadır.

Örnek: Web sitesine "Nasıl Kurulur?" ve "Uygulama Özellikleri" gibi eğitici videolar yüklenir. Müşterilerin sorularını sorabileceği bir destek e-posta adresi oluşturulur.

Servis, Bakım

2

Satış sonrası destektir. Garanti kapsamındaki onarımlar, arızalı ürün değişimi, müşteri sorularına cevap verilmesi ve en önemlisi, ürünü güncel tutmak için düzenli yazılım güncellemelerinin yayınlanması bu sürecin parçasıdır.

Örnek: Bir müşteri cihazının internete bağlanmadığını bildirir. Destek ekibi sorunu uzaktan inceler ve bir yazılım hatası olduğunu tespit eder. Tüm kullanıcılara bu hatayı düzelten bir mobil uygulama güncellemesi gönderilir.

Rekabete Karşı Direnme, Destek Sürekliliği

3

Pazar dinamiktir ve rakipler sürekli yeni ürünler çıkarır. Bu adım, ürünün pazardaki değerini koruması ve artırması için yapılan sürekli çabayı ifade eder. Müşteri geri bildirimlerini dinleyerek yeni özellikler eklemek, ürünü teknolojik olarak güncel tutmak ve markaya olan güveni sürdürmek esastır.

Örnek: Bir rakip firma, sistemine gübreleme özelliği ekler. Buna karşılık, bizim Ar-Ge ekibimiz "Akıllı Sulama Sistemi v2.0" için sıvı gübre ekleyebilen bir modül üzerinde çalışmaya başlar. Mevcut müşterilere ise sadakati artırmak için yeni yazılım özellikleri (örn. detaylı su tüketim raporu) ücretsiz olarak sunulur.

b.2) Yazılımı Etkileyen Modern Eğilimler

1. Yaygınlaşan Bilgi-İşlem (Ubiquitous Computing)

Bu eğilim, işlem gücünün (CPU, bellek vb.) sadece bilgisayar ve telefonlarda değil, hayatımızdaki hemen her nesnenin içine gömülecek kadar küçülmesi ve ucuzlaması anlamına gelir. "Her yerde hesaplama" felsefesiyle, gündelik eşyalar "akıllı" hale gelir ve bu da onlar için özel yazılımlar geliştirme ihtiyacını doğurur.

Örnek: Akıllı saatler bu eğilimin en iyi örneklerindendir. Küçük bir saat kasasının içinde güçlü bir işlemci, sensörler ve bir işletim sistemi bulunur. Bu durum yazılım geliştiriciler için yeni zorluklar ve fırsatlar yaratır: Yazılımın çok daha az pil tüketmesi, küçük bir ekranda etkili bir şekilde çalışması (UI/UX tasarımı) ve giyilebilir bir cihazın sınırlı donanım kaynaklarını en verimli şekilde kullanması gerekir. Benzer şekilde, akıllı buzdolapları, akıllı aydınlatma sistemleri veya arabalardaki bilgi-eğlence sistemleri de bu eğilimin birer ürünüdür.

2. Yaygınlaşan Haberleşme Ağı (Ubiquitous Network)

Bu eğilim, bir öncekiyle paralel olarak, cihazların neredeyse her zaman ve her yerde internete (Wi-Fi, 5G, Bluetooth vb.) bağlı olabilmesidir. Yazılımlar artık tek bir cihazda izole bir şekilde çalışmak yerine, sürekli olarak veri alıp gönderebilecekleri varsayımıyla tasarlanır. Bu durum, bulut tabanlı hizmetlerin ve Nesnelerin İnterneti'nin (IoT) temelini oluşturur.

Örnek: Bulut tabanlı ofis uygulamaları (Google Docs, Microsoft 365) bu eğilimi mükemmel bir şekilde kullanır. Bir kullanıcı, dizüstü bilgisayarında bir belge üzerinde çalışmaya başlayabilir, yolda telefonundan (5G ile) aynı belgeye devam edebilir ve evdeki tabletinden (Wi-Fi ile) son düzenlemeleri yapabilir. Yazılım, cihazlar arasında veriyi sürekli senkronize eder çünkü kesintisiz bir ağ bağlantısına güvenir. Bir önceki örnekteki akıllı sulama sisteminin hava durumu verilerini anlık olarak internetten çekmesi de bu eğilim sayesinde mümkündür.

b.2) Yazılımı Etkileyen Modern Eğilimler

3. Özgür / Açık Kaynak Yazılım (Free / Open Source Software)

Bu, yazılımın kaynak kodlarının herkes tarafından görüntülenebildiği, değiştirilebildiği ve dağıtılabildiği bir geliştirme modelidir. Dünya çapında binlerce bağımsız geliştiricinin gevşek bir organizasyonla iş birliği yapmasına olanak tanır. Bu kadar çok kişinin aynı proje üzerinde çalışabilmesi için yazılan kodun son derece anlaşılır, temiz ve iyi belgelenmiş olması bir zorunluluk haline gelir. Bu model, inovasyonu hızlandırır ve yazılım geliştirme maliyetlerini düşürür.

Örnek: Bugün kullandığımız teknolojilerin büyük bir kısmı açık kaynak kodludur. Android işletim sisteminin çekirdeği, web sunucularının çoğunda çalışan Linux, web tarayıcısı Firefox ve veri biliminde yaygın olarak kullanılan Python dili bu modelin ürünleridir. Bir şirket, sıfırdan bir veritabanı yazmak yerine, açık kaynak kodlu ve milyonlarca kişi tarafından test edilip geliştirilmiş PostgreSQL'i kullanarak hem zamandan hem de maliyetten tasarruf eder.

4. Takım Çalışması Zorunluluğu

Modern bir yazılım projesi (örneğin bir bankacılık uygulaması, bir sosyal medya platformu veya bir video oyunu) tek bir kişinin altından kalkamayacağı kadar karmaşıktır. Bu nedenle projeler, farklı uzmanlıklara sahip kişilerden oluşan takımlar tarafından yürütülür: Arayüz geliştiricileri, veritabanı uzmanları, sistem mimarları, test mühendisleri, proje yöneticileri vb. Bu zorunluluk, Agile ve Scrum gibi çevik proje yönetim metodolojilerinin ve iş birliği araçlarının doğmasına neden olmuştur.

Örnek: Bir e-ticaret sitesi geliştiren yazılım ekibini düşünelim. Bu ekip, işlerini yönetmek için Jira veya Trello gibi bir proje yönetim aracı kullanır. Her sabah 15 dakikalık "stand-up" toplantılarıyla kimin ne üzerinde çalıştığını ve ne gibi zorluklar yaşadığını paylaşır. Birden fazla geliştiricinin aynı kod tabanı üzerinde çakışma yaşamadan çalışabilmesi için Git gibi bir versiyon kontrol sistemi kullanmak zorundadırlar.

b.2) Yazılımı Etkileyen Modern Eğilimler

5. Küreselleşme

Küreselleşme, yazılım dünyasını iki temel yönde etkiler: 1) Geliştirme Takımları: Ekipler artık aynı ofiste olmak zorunda değildir; dünyanın farklı ülkelerine ve zaman dilimlerine dağılmış olabilirler. 2) Hedef Kitle: Yazılımlar artık sadece yerel bir pazar için değil, küresel bir kitle için tasarlanır.

- Örnek: Bir yazılım şirketinin merkezi ABD'de olabilirken, geliştirme ekibinin bir kısmı Hindistan'da, tasarım ekibi Polonya'da ve müşteri desteği ekibi Mısır'da bulunabilir. Bu küresel takım, Slack ve Zoom gibi iletişim araçlarıyla farklı zaman dilimlerinde verimli bir şekilde çalışmak zorundadır. Aynı zamanda, geliştirdikleri uygulama (örneğin Spotify) farklı dilleri (uluslararasılaştırma), para birimlerini ve kültürel tercihleri (yerelleştirme) desteklemek zorundadır.

6. Ekonomik Krizler

Modern bir yazılım projesi (örneğin bir bankacılık uygulaması, bir sosyal medya platformu veya bir video oyunu) tek bir kişinin altından kalkamayacağı kadar karmaşıktır. Bu nedenle projeler, farklı uzmanlıklara sahip kişilerden oluşan takımlar tarafından yürütülür: Arayüz geliştiricileri, veritabanı uzmanları, sistem mimarları, test mühendisleri, proje yöneticileri vb. Bu zorunluluk, Agile ve Scrum gibi çevik proje yönetim metodolojilerinin ve iş birliği araçlarının doğmasına neden olmuştur.

Örnek: Ekonomik krizdeki bir perakende şirketi, "artırılmış gerçeklik ile sanal ürün deneme" gibi yenilikçi ama pahalı bir projeyi erteleyebilir. Bunun yerine, tüm yazılım bütçesini "online mağazanın ödeme sayfasını iyileştirerek sepet terk oranını %3 düşürme" gibi doğrudan satışları artıracak bir projeye yönlendirebilir. Ayrıca, kriz dönemlerinde şirketlerin büyük tek seferlik lisans ücretleri ödemek yerine, SaaS (Hizmet Olarak Yazılım) modeline (örneğin Adobe Creative Cloud, Salesforce) yönelmesi yaygınlaşır. Bu model, yazılım şirketlerini abonelik tabanlı gelir modelleri geliştirmeye iter.